PLAN DE PRUEBA

Sistema de Gestión de Órdenes (OMS)

Estilo Amazon — Vacante Senior

Versión	1.0
Fecha	Abril 2026
Tipo de documento	Plan de Prueba — Vacante Senior
Estado	Aprobado para uso en proceso de selección
Clasificación	**Confidencial**

1. Introducción

El presente documento describe el Plan de Prueba para el Sistema de Gestión de Órdenes (OMS) estilo Amazon, desarrollado como parte de una evaluación técnica para la vacante de Desarrollador Senior. El plan cubre pruebas unitarias y de seguridad (JWT/Auth) sobre los módulos principales del sistema.

1.1 Propósito

Validar que el sistema cumple con los requisitos funcionales y de seguridad definidos, garantizando la integridad de los datos, la protección de los endpoints y la correcta gestión del ciclo de vida de las órdenes.

1.2 Alcance

- Registro y autenticación de usuarios (JWT/OAuth)
- Gestión de productos y categorías
- Creación y gestión de órdenes con múltiples productos
- Validación de stock antes de confirmar
- Gestión de estados de la orden
- Registro de pagos simulados
- Auditoría e historial
- Control de concurrencia
- Notificaciones por correo (opcional)

1.3 Fuera de alcance

- Pruebas de integración contra pasarelas de pago reales
- Pruebas E2E de interfaz gráfica Angular
- Pruebas de rendimiento / carga

2. Stack Tecnológico

El sistema se construye sobre el siguiente stack, que el candidato deberá dominar en su totalidad:

Capa	Tecnología	Versión / Detalle
Backend — Lenguaje	Java	21 LTS
Backend — Framework	Spring Boot	3.x o 4.x

Capa	Tecnología	Versión / Detalle
Backend — Persistencia	Spring Data JPA	Incluido en Spring Boot
Backend — Seguridad	JWT / OAuth2	Spring Security 6.x
Build — Backend	Maven	3.9.x
Base de datos	SQL Server	2019 o última versión
Frontend	Angular	19 o última versión
Frontend — Estilos	Tailwind CSS	3.x o 4.x
Arquitectura	Hexagonal + Microservicios	DDD recomendado
Contenedores	Docker / Docker Compose	Opcional — valorado

3. Contexto del Negocio

El OMS es un sistema que permite a los usuarios registrados crear órdenes con múltiples productos, validar stock en tiempo real antes de confirmar, gestionar el ciclo completo de estados de la orden, registrar pagos simulados, mantener un historial de auditoría completo y evitar inconsistencias derivadas de operaciones concurrentes.

3.1 Flujo Principal de la Orden

El ciclo de vida de una orden sigue el siguiente flujo de estados:

Estado	Nombre	Descripción
1	PENDIENTE	Orden creada, aún no confirmada ni pagada.
2	CONFIRMADA	Stock validado y reservado exitosamente.
3	PAGADA	Pago simulado registrado correctamente.
4	ENVIADA	Orden despachada al usuario.
5	CANCELADA	Orden cancelada. Stock liberado.

4. Tareas Principales del Sistema

El candidato debe implementar y probar los siguientes módulos funcionales:

- **Registro de Usuarios — CRUD completo con validaciones estrictas.**
  - Nombre, email único, contraseña segura, rol válido.
- **Registro de Productos — Alta, modificación y baja de productos.**
  - Nombre, precio > 0, stock >= 0, categoría válida.
- **Creación de Órdenes con múltiples productos — Lógica transaccional.**
  - Validación de stock por ítem. Cálculo de totales.
- **Validación de Stock — Verificar disponibilidad antes de confirmar.**
  - Manejo de concurrencia (Optimistic Locking recomendado).
- **Gestión de Estados de la Orden — Máquina de estados.**
  - Transiciones válidas únicamente. Auditoría de cada cambio.
- **Registro de Pagos Simulados — Sin pasarela real.**
  - Referencia externa, método de pago, estado del pago.
- **Historial / Auditoría — Log de todas las operaciones.**
  - Tabla AUDIT_LOG: quién, qué entidad, qué operación, datos antes/después.
- **Control de Concurrencia — Evitar condiciones de carrera en stock.**
  - Optimistic Locking (campo 'version') o Pessimistic Locking según criterio.
- **Reportes de Órdenes — Consultas por usuario, estado, fecha.**

- **Notificaciones por Correo — Opcional, valorado positivamente.**
  - JavaMailSender o servicio externo (SendGrid, Mailgun).

5. Modelo Entidad-Relación (MER)

El siguiente MER describe las entidades principales del sistema. El candidato puede agregar, modificar o extender el modelo según su criterio técnico (ej. tabla ROLES, ADDRESSES, STOCK_MOVEMENTS, soft delete con deleted_at, etc.).

5.1 Entidades Principales

Entidad	PK	Campos Principales
USERS	id (bigint)	nombre, email, password_hash, rol, activo, created_at, updated_at
CATEGORIES	id (bigint)	nombre, descripcion, created_at
PRODUCTS	id (bigint)	nombre, precio, stock, category_id (FK), activo, created_at, updated_at
ORDERS	id (bigint)	user_id (FK), estado, total, fecha_orden, fecha_actualizacion
ORDER_ITEMS	id (bigint)	order_id (FK), product_id (FK), cantidad, precio_unitario, subtotal
PAYMENTS	id (bigint)	order_id (FK), metodo_pago, estado_pago, monto, referencia_externa, fecha_pago
AUDIT_LOG	id (bigint)	user_id (FK), tabla_afectada, operacion, datos_anteriores, datos_nuevos, fecha_evento
NOTIFICATIONS	id (bigint)	user_id (FK), order_id (FK), tipo, estado, mensaje, fecha_envio

5.2 Relaciones Principales

- USERS (1) ——< ORDERS (N): Un usuario puede tener múltiples órdenes.
- ORDERS (1) ——< ORDER_ITEMS (N): Una orden contiene uno o más ítems.
- PRODUCTS (1) ——< ORDER_ITEMS (N): Un producto puede aparecer en múltiples ítems de orden.
- CATEGORIES (1) ——< PRODUCTS (N): Una categoría agrupa múltiples productos.
- ORDERS (1) ——o| PAYMENTS (0..1): Una orden tiene como máximo un pago registrado.
- ORDERS (1) ——< NOTIFICATIONS (N): Una orden puede disparar múltiples notificaciones.
- USERS (1) ——< AUDIT_LOG (N): Un usuario genera múltiples entradas de auditoría.

6. Reglas de Validación

Las siguientes validaciones deben ser implementadas tanto en el backend (capa de dominio/aplicación) como verificadas mediante pruebas unitarias.

6.1 Usuarios

Campo	Regla	Mensaje de Error Esperado
Nombre	No puede estar vacío	El nombre es obligatorio.
Email	Formato válido y único en el sistema	El email debe ser válido y no estar registrado.
Contraseña	Min. 8 caracteres, 1 mayúscula, 1 minúscula, 1 número	La contraseña no cumple los requisitos de seguridad.
Rol	Valor válido del enum de roles definido	El rol indicado no es válido.

6.2 Productos

Campo	Regla	Mensaje de Error Esperado
Nombre	No puede estar vacío	El nombre del producto es obligatorio.
Precio	Debe ser un valor mayor a 0	El precio debe ser mayor a cero.
Stock	Debe ser mayor o igual a 0	El stock no puede ser negativo.
Categoría	Debe existir en el sistema y ser válida	La categoría indicada no existe o no es válida.

6.3 Órdenes

Campo	Regla	Mensaje de Error Esperado
Usuario	Debe estar registrado y activo	El usuario no existe o está inactivo.
Productos	Lista no vacía; cada producto debe existir	La orden debe contener al menos un producto válido.
Total	Calculado automáticamente; debe ser > 0	El total de la orden debe ser mayor a cero.
Estado	Transiciones válidas según máquina de estados	La transición de estado solicitada no es válida.
Stock	Suficiente disponible al momento de confirmar	Stock insuficiente para el producto indicado.

6.4 Notificaciones (Opcional)

Campo	Regla	Mensaje de Error Esperado
Usuario	Debe existir y estar activo	El usuario destinatario no es válido.
Orden	Debe existir y pertenecer al usuario	La orden asociada no es válida.
Tipo	Valor del enum: CONFIRMACION, PAGO, ENVIO, etc.	El tipo de notificación no es válido.
Estado	Valor del enum: PENDIENTE, ENVIADA, FALLIDA	El estado de notificación no es válido.

7. Plan de Pruebas Unitarias

Las pruebas unitarias deben implementarse con JUnit 5 + Mockito para el backend (Java 21 / Spring Boot) y con Jasmine / Jest para el frontend (Angular 19). Cada caso de prueba es independiente y no requiere infraestructura real (base de datos, servidor de correo, etc.).

7.1 Herramientas y Configuración

Herramienta	Propósito	Versión Recomendada
JUnit 5 (Jupiter)	Framework de pruebas backend	5.10.x (incluido en Spring Boot)
Mockito	Mocking de dependencias	5.x (incluido en Spring Boot)
Spring Boot Test	Contexto de prueba ligero (@DataJpaTest, @WebMvcTest)	Mismo que Spring Boot
AssertJ	Aserciones fluidas	3.x (incluido en Spring Boot)
H2 Database	Base de datos en memoria para @DataJpaTest	2.x
Jasmine / Jest	Pruebas unitarias Angular	Jasmine 4 / Jest 29
Angular TestBed	Pruebas de componentes Angular	19.x
Jacoco	Cobertura de código backend	0.8.x

7.2 Módulo: Registro de Usuario

Clase bajo prueba: UserService / UserDomainValidator

ID	Caso de Prueba	Entrada	Resultado Esperado	Prioridad
UT-US-01	Registro exitoso de usuario	Datos válidos completos	Usuario creado, ID generado, contraseña hasheada	Alta
UT-US-02	Email duplicado	Email ya existente en BD	Excepción EmailAlreadyExistsException	Alta
UT-US-03	Nombre vacío	nombre = ""	ConstraintViolationException / Bad Request	Alta
UT-US-04	Email inválido	email = "noesmail"	Formato de email inválido	Alta

ID	Caso de Prueba	Entrada	Resultado Esperado	Prioridad
UT-US-05	Contraseña sin mayúscula	password = "abc12345"	Contraseña no cumple política de seguridad	Alta
UT-US-06	Contraseña sin número	password = "Abcdefgh"	Contraseña no cumple política de seguridad	Alta
UT-US-07	Contraseña menor a 8 caracteres	password = "Ab1"	Contraseña no cumple longitud mínima	Alta
UT-US-08	Rol inválido	rol = "SUPERADMIN"	Rol no reconocido por el sistema	Media
UT-US-09	Nombre con espacios en blanco	nombre = " "	El nombre es obligatorio (no solo espacios)	Media

7.3 Módulo: Gestión de Productos

Clase bajo prueba: ProductService / ProductDomainValidator

ID	Caso de Prueba	Entrada	Resultado Esperado	Prioridad
UT-PR-01	Creación exitosa de producto	Datos válidos completos	Producto creado con ID asignado	Alta
UT-PR-02	Nombre de producto vacío	nombre = ""	El nombre del producto es obligatorio	Alta
UT-PR-03	Precio igual a 0	precio = 0	El precio debe ser mayor a cero	Alta
UT-PR-04	Precio negativo	precio = -5.00	El precio debe ser mayor a cero	Alta
UT-PR-05	Stock negativo	stock = -1	El stock no puede ser negativo	Alta
UT-PR-06	Stock igual a 0 (válido)	stock = 0	Producto creado correctamente (stock 0 es válido)	Alta
UT-PR-07	Categoría inexistente	category_id = 9999	La categoría indicada no existe	Media

ID	Caso de Prueba	Entrada	Resultado Esperado	Prioridad
UT-PR-08	Actualizar stock: incremento	stock actual 10, suma 5	Stock resultante = 15	Alta
UT-PR-09	Actualizar stock: decremento	stock actual 3, resta 5	Stock resultante negativo — rechazar	Alta

7.4 Módulo: Gestión de Órdenes

Clase bajo prueba: OrderService / OrderDomainValidator / OrderStateMachine

ID	Caso de Prueba	Entrada	Resultado Esperado	Prioridad
UT-OR-01	Crear orden con un producto	Usuario activo, producto con stock >= 1	Orden en estado PENDIENTE creada	Alta
UT-OR-02	Crear orden con múltiples productos	3 productos distintos con stock suficiente	Orden con 3 ítems y total calculado correcto	Alta
UT-OR-03	Confirmar orden con stock suficiente	Orden PENDIENTE, stock >= cantidad	Orden pasa a CONFIRMADA, stock decrementado	Alta
UT-OR-04	Confirmar orden con stock insuficiente	Orden PENDIENTE, stock < cantidad requerida	Stock insuficiente — orden permanece PENDIENTE	Alta
UT-OR-05	Orden sin productos	Lista de ítems vacía	La orden debe contener al menos un producto	Alta
UT-OR-06	Calcular total de orden	Ítem A: $10 x 2 + Ítem B: $5 x 3	Total = $35.00	Alta
UT-OR-07	Transición PENDIENTE → CONFIRMADA	Orden en estado PENDIENTE	Cambio de estado exitoso	Alta
UT-OR-08	Transición CONFIRMADA → PAGADA	Orden en estado CONFIRMADA + pago registrado	Cambio de estado exitoso	Alta
UT-OR-09	Transición inválida PAGADA → PENDIENTE	Orden en estado PAGADA	Transición no permitida — excepción	Alta
UT-OR-10	Cancelar orden PENDIENTE	Orden en estado PENDIENTE	Orden CANCELADA, stock liberado	Alta
UT-OR-11	Cancelar orden PAGADA	Orden en estado PAGADA	Transición no permitida o requiere flujo de reembolso	Media
UT-OR-12	Concurrencia: dos órdenes simultáneas mismo producto	Stock = 1, dos requests concurrentes	Solo una orden confirmada, la otra rechazada	Alta

7.5 Módulo: Pagos Simulados

Clase bajo prueba: PaymentService

ID	Caso de Prueba	Entrada	Resultado Esperado	Prioridad
UT-PA-01	Registrar pago exitoso	Orden CONFIRMADA, monto correcto	Pago registrado, orden pasa a PAGADA	Alta
UT-PA-02	Pagar orden no confirmada	Orden en estado PENDIENTE	No se permite pagar sin confirmar	Alta
UT-PA-03	Pago duplicado en misma orden	Orden ya PAGADA, segundo intento de pago	No se permite registro doble de pago	Alta
UT-PA-04	Monto incorrecto	Monto distinto al total de la orden	Pago rechazado por discrepancia de monto	Media

7.6 Módulo: Auditoría

Clase bajo prueba: AuditService / AuditListener

ID	Caso de Prueba	Entrada	Resultado Esperado	Prioridad
UT-AU-01	Crear usuario genera log	Usuario creado exitosamente	Entrada en AUDIT_LOG con operación INSERT	Alta
UT-AU-02	Cambio de estado de orden genera log	Transición de estado válida	Entrada en AUDIT_LOG con estado anterior y nuevo	Alta
UT-AU-03	Log contiene datos anteriores y nuevos	Cualquier operación UPDATE	JSON con datos_anteriores y datos_nuevos poblados	Media
UT-AU-04	Log identifica al usuario responsable	Cualquier operación autenticada	user_id del JWT está registrado en el log	Alta

8. Pruebas de Seguridad — JWT / OAuth2

Las pruebas de seguridad validan que los endpoints estén correctamente protegidos, que los tokens JWT sean procesados de forma segura y que el sistema maneje adecuadamente los casos de autenticación y autorización inválidos.

8.1 Configuración de Seguridad Esperada

- Todos los endpoints protegidos requieren token JWT válido en el header Authorization: Bearer <token>.
- Endpoints públicos (login, registro) accesibles sin autenticación.
- Roles diferenciados: ADMIN puede gestionar productos y categorías; USER solo puede crear/consultar sus órdenes.
- Tokens con tiempo de expiración configurado (ej. 1 hora).
- Refresh token implementado (valorado positivamente).

8.2 Casos de Prueba — Autenticación

ID	Caso de Prueba	Entrada	Resultado Esperado	HTTP Code
SEC-AU-01	Login con credenciales válidas	email/password correctos	Token JWT generado y retornado	200 OK
SEC-AU-02	Login con contraseña incorrecta	Password erróneo	Acceso denegado, sin token	401 Unauthorized
SEC-AU-03	Login con usuario inexistente	Email no registrado	Usuario no encontrado	401 Unauthorized
SEC-AU-04	Login con usuario inactivo	Usuario con activo = false	Cuenta inactiva, acceso denegado	401 Unauthorized
SEC-AU-05	Acceso a endpoint protegido sin token	Request sin Authorization header	Acceso rechazado	401 Unauthorized
SEC-AU-06	Token JWT malformado	Authorization: Bearer abc123	Token inválido rechazado	401 Unauthorized
SEC-AU-07	Token JWT expirado	Token con exp en el pasado	Token expirado, requiere nuevo login	401 Unauthorized
SEC-AU-08	Token JWT con firma alterada	Token con signature modificado	Firma inválida, acceso denegado	401 Unauthorized

8.3 Casos de Prueba — Autorización por Rol

ID	Caso de Prueba	Entrada	Resultado Esperado	HTTP Code
SEC-RO-01	ADMIN crea producto	Token con rol ADMIN	Producto creado exitosamente	201 Created
SEC-RO-02	USER intenta crear producto	Token con rol USER	Acceso prohibido al recurso	403 Forbidden
SEC-RO-03	USER crea su propia orden	Token con rol USER válido	Orden creada exitosamente	201 Created
SEC-RO-04	USER consulta órdenes de otro usuario	user_id distinto al del token	Acceso prohibido o recurso no encontrado	403 / 404
SEC-RO-05	ADMIN accede a log de auditoría	Token con rol ADMIN	Log retornado correctamente	200 OK
SEC-RO-06	USER accede a log de auditoría	Token con rol USER	Acceso prohibido	403 Forbidden
SEC-RO-07	ADMIN consulta cualquier orden	Token ADMIN + order_id de otro usuario	Orden retornada (acceso total para ADMIN)	200 OK

8.4 Casos de Prueba — Seguridad de Datos

ID	Caso de Prueba	Entrada	Resultado Esperado	HTTP Code
SEC-DA-01	Contraseña almacenada con hash	Consulta directa a BD	Campo password_hash nunca en texto plano (BCrypt)	N/A — Verificación de BD
SEC-DA-02	Contraseña no expuesta en respuesta API	GET /users/{id}	password_hash omitido del JSON de respuesta	200 OK
SEC-DA-03	JWT no contiene datos sensibles	Decodificar payload del token	Payload sin password, sin datos bancarios	N/A — Verificación de token
SEC-DA-04	Headers de seguridad presentes	Cualquier respuesta HTTP	X-Content-Type-Options, X-Frame-Options presentes	N/A — Verificación de headers

9. Criterios de Aceptación

El candidato deberá cumplir con los siguientes criterios para considerar el plan de prueba como exitoso:

Criterio	Detalle
Cobertura mínima	80% de cobertura de líneas en servicios de dominio (Jacoco).
Pruebas unitarias exitosas	100% de los casos de prueba definidos pasan (estado PASS).
Seguridad JWT	Todos los casos SEC-AU y SEC-RO pasan correctamente.
Validaciones	Todos los UT de validación retornan el mensaje de error correcto.
Concurrencia	UT-OR-12 pasa consistentemente en múltiples ejecuciones.
Auditoría	Todo cambio de estado y creación de entidad genera entrada en AUDIT_LOG.
Arquitectura Hexagonal	Clases de dominio sin dependencias de frameworks (Spring, JPA) en el core.
Docker (Opcional)	docker-compose up levanta toda la infraestructura sin configuración adicional.

10. Consideraciones de Arquitectura Hexagonal

El sistema debe seguir los principios de Arquitectura Hexagonal (Ports & Adapters). Se evaluará la correcta separación de capas:

Capa	Responsabilidad	Tecnología Correspondiente
Domain (Core)	Entidades, Value Objects, reglas de negocio puras, puertos (interfaces)	Java puro — sin Spring, sin JPA
Application	Casos de uso (Use Cases / Services), orquestación del dominio	Spring @Service — inyección por constructor
Infrastructure — Adapters In	Controllers REST, excepciones HTTP, DTOs de entrada	Spring MVC / WebFlux, Jackson
Infrastructure — Adapters Out	Repositorios JPA, entidades JPA, adaptadores de correo, etc.	Spring Data JPA, SQL Server, JavaMailSender
Infrastructure — Config	Configuración de seguridad, JWT, beans, Docker	Spring Security, Docker Compose

Se penalizará si se detectan dependencias de frameworks en el núcleo de dominio (ej. @Entity en clases de dominio, acceso a repositorios desde el dominio directamente).

11. Entregables del Candidato

El candidato deberá entregar los siguientes artefactos para la evaluación:

- **Repositorio Git (GitHub / GitLab / Bitbucket) con el código fuente completo.**
  - README.md con instrucciones de instalación y ejecución.
  - Commits descriptivos y organizados (se valora uso de Conventional Commits).
- **Proyecto backend Spring Boot estructurado con Arquitectura Hexagonal.**
  - Maven como gestor de dependencias (pom.xml completo).
  - Tests unitarios en src/test/ con cobertura documentada.
- **Proyecto frontend Angular 19 funcional.**
  - Tailwind CSS como sistema de estilos.
  - Consumo de los endpoints del backend implementados.
- **Scripts SQL o Migraciones (Flyway / Liquibase recomendado).**
  - Scripts de creación de tablas para SQL Server 2019.
  - Datos de prueba (seed) para demostración.
- **Reporte de cobertura Jacoco (formato HTML o XML).**
- **docker-compose.yml funcional (Opcional — valorado positivamente).**
- **Colección Postman / OpenAPI (Swagger) para documentación de endpoints.**

12. Anexo — Resumen de Casos de Prueba

Módulo	Total de Casos	Prioridad Alta	Cobertura Objetivo
Usuarios	9 casos (UT-US-01 a 09)	7 casos	Domain + Service
Productos	9 casos (UT-PR-01 a 09)	7 casos	Domain + Service
Órdenes	12 casos (UT-OR-01 a 12)	11 casos	Domain + StateMachine
Pagos	4 casos (UT-PA-01 a 04)	3 casos	Domain + Service
Auditoría	4 casos (UT-AU-01 a 04)	3 casos	Listener + Service
Seguridad — Auth	8 casos (SEC-AU-01 a 08)	8 casos	Security Config + JWT Filter
Seguridad — Roles	7 casos (SEC-RO-01 a 07)	7 casos	Authorization + Controller
Seguridad — Datos	4 casos (SEC-DA-01 a 04)	4 casos	Encoder + Serializer
TOTAL	57 casos de prueba	50 prioridad alta	Cobertura mínima 80%

Documento generado para proceso de selección — Vacante Desarrollador Senior OMS